



MKD

2026-06-20

1 Contest

2 Number theory

Contest (1)

dbg.h20 lines

```
#define dbg(...) cerr<<"DEBUG:["<<#__VA_ARGS__<<"] = ["\n\n"
    _print(__VA_ARGS__),cerr<<"\n\n"
template<class T, class=void>struct cntnr : false_type {};
template<class T>struct cntnr<T,
    void_t<decltype(declval<T>()).begin()),
    decltype(declval<T>().end()),
    typename T::value_type> > : true_type {};
template<class T>using raw=remove_cv_t<remove_reference_t<T> >;
template<class T>enable_if_t<!cntnr<T>::value>
    _print(const T &x) { cerr << x; }
template<class T>enable_if_t<cntnr<T>::value>_print(const T &v){
    cerr << "["; bool f = false;
    for (const auto &x: v)
        f ? cerr << (cntnr<raw<decltype(x)> >::value ?",\n":", "):
            cerr, f = true, _print(x);
    cerr << "]"
}
template<class T, class ... A>
void _print(const T &x, const A &... a) {
    _print(x);(cerr << " ", " ", _print(a)), ... );}
```

template.cpp25 lines

```
#include <bits/stdc++.h>
using namespace std;
// #define LOCAL_DEBUG
#ifdef LOCAL_DEBUG
#include "dbg.h"
#else
#define dbg(...) 42
#endif
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
using ll = long long;
using pii = pair<int, int>;
using vi = vector<int>;
void init() {}
void solve() {}
signed main() {
    init();
    cin.tie(nullptr), ios::sync_with_stdio(false);
    // cin.exceptions(cin.failbit);
    int T = 1;
    // cin >> T;
    while (T-->solve();
    return 0;
}
```

1.1 checker

cek.sh19 lines

```
#!/usr/bin/env bash
set -u
cd "$(dirname "$0")"
g++ gen.cpp -o gen -std=c++17 -O2
g++ std.cpp -o std -std=c++17 -O2
g++ sol.cpp -o sol -std=c++17 -O2
cnt=0
```

```
while true; do
    cnt=$((cnt + 1))
    echo "Testing case #$cnt"
    ./gen "$1-" > in.txt
    ./std < in.txt > out1.txt
    ./sol < in.txt > out2.txt
    if ! diff -q out1.txt out2.txt > /dev/null; then
        echo "WA on case #$cnt"
        diff out1.txt out2.txt
        read -r -p "Press Enter to continue ..."
    fi
done
```

cek.bat19 lines

```
@echo off
cd "%~dp0"
g++ gen.cpp -o gen -std=c++17 -O2
g++ std.cpp -o std -std=c++17 -O2
g++ sol.cpp -o sol -std=c++17 -O2
set cnt=0
:loop
set /a cnt+=1
echo Testing case %cnt%
gen %1 > in.txt
std < in.txt > out1.txt
sol < in.txt > out2.txt
fc out1.txt out2.txt > nul
if errorlevel 1 (
    echo WA on case %cnt%
    fc out1.txt out2.txt
    pause
)
goto loop
```

gen.h22 lines

```
uint64_t seed = chrono::steady_clock::now()
    .time_since_epoch().count() ^ random_device {} ();
mt19937 rng(seed);
template<class T>
T randi(T l, T r){return uniform_int_distribution<T>(l,r)(rng); }
vi randvi(int n, int l, int r) {
    vi v(n);
    rep(i, 0, sz(v))v[i] = randi(l, r);
    return v;
}
vi randp(int n) {
    vi p(n);
    iota(all(p), 1);
    shuffle(all(p), rng);
    return p;
}
int main(int argc, char *argv[]) {
    if (argc >= 2)
        seed = stoull(argv[1]), rng.seed(seed);
    cerr << seed << endl;
    return 0;
}
```

Number theory (2)

2.1 模运算

ModularArithmetic.h2f9ed0, 18 lines

```
Description: Operators for modular arithmetic. You need to set mod to
some number first and then you can use the structure.
"euclid.h"
```

constexpr ll MOD = 17; // change to something else
struct Mod {
 ll x;
 Mod(ll _x) : x((_x % MOD + MOD) % MOD) {}
 Mod operator+(Mod b) { return Mod(((x + b.x)%MOD+MOD)%MOD); }
 Mod operator-(Mod b) { return Mod(((x - b.x)%MOD+MOD)%MOD); }
 Mod operator*(Mod b) { return Mod((x * b.x %MOD+MOD)%MOD); }
 Mod operator/(Mod b) { return *this * invert(b); }
 Mod invert(Mod a) {
 ll x, y, g = euclid(a,x, MOD, x, y);
 assert(g == 1);return Mod((x % MOD + MOD) % MOD);
 }
 Mod operator^(ll e) {
 if (!e) return Mod(1);
 Mod r = *this ^ (e / 2); r = r * r;
 return e & 1 ? *this * r : r;
 }
};

模逆元

$$\frac{a}{b} \equiv a \cdot b_{\text{inv}} \equiv x \pmod{p}$$
$$\Rightarrow a \equiv bx \pmod{p}$$
$$\Rightarrow a \cdot b_{\text{inv}} \cdot b \equiv bx \pmod{p}$$
$$\Rightarrow a \cdot (b_{\text{inv}} \cdot b - 1) \equiv 0 \pmod{p}$$
$$\Rightarrow b_{\text{inv}} \cdot b \equiv 1 \pmod{p}$$
$$\text{inv存在} \Leftrightarrow \gcd(b, p) = 1$$
$$b \cdot b_{\text{inv}} = k \cdot p + 1, g = \gcd(b, p) \Rightarrow g \cdot b_1 \cdot b_{\text{inv}} = k \cdot g \cdot p_1 + 1$$
$$\Rightarrow g \cdot (b_1 \cdot b_{\text{inv}} - k \cdot p_1) = 1$$
$$\text{数域为}\mathbb{Z} \Rightarrow g = 1, b_1 \cdot b_{\text{inv}} - k \cdot p_1 = 1$$

ModInverse.h

Description: Pre-computation of modular inverses. Assumes LIM ≤ mod and that mod is a prime.6f684f, 3 lines

```
const ll mod = 1000000007, LIM = 200000;
ll* inv = new ll[LIM] - 1; inv[1] = 1;
rep(i,2,LIM) inv[i] = mod - (mod / i) * inv[mod % i] % mod;
```

ModPow.h

const ll mod = 1000000007; // faster if constb83e45, 8 lines

```
ll modpow(ll b, ll e) {
    ll ans = 1;
    for (; e; b = b * b % mod, e /= 2)
        if (e & 1) ans = ans * b % mod;
    return ans;
}
```

ModLog.h

Description: Returns the smallest x > 0 s.t. a^x = b (mod m), or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a. Time: O(√m)c040b8, 11 lines

```
ll modLog(ll a, ll b, ll m) {
    ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<ll, ll> A;
    while (j <= n && (e = f = e * a % m) != b % m)
        A[e * b % m] = j++;
}
```

```
if (e == b % m) return j;
if (__gcd(m, e) == __gcd(m, b))
    rep(i,2,n+2) if (A.count(e = e * f % m))
        return n * i - A[e];
return -1;
}
```

ModSum.h
Description: Sums of mod'ed arithmetic progressions.
 $\text{modsum}(\text{to}, c, k, m) = \sum_{i=0}^{\text{to}-1} (ki + c) \% m$. **divsum** is similar but for floored division.
Time: $\log(m)$, with a large constant.

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
```

```
ull divsum(ull to, ull c, ull k, ull m) {
    ull res = k / m * sumsq(to) + c / m * to;
    k %= m; c %= m;
    if (!k) return res;
    ull to2 = (to * k + c) / m;
    return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}
```

```
ll modsum(ull to, ll c, ll k, ll m) {
    c = ((c % m) + m) % m;
    k = ((k % m) + m) % m;
    return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h
Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.
Time: $\mathcal{O}(1)$ for **modmul**, $\mathcal{O}(\log b)$ for **modpow**

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
    ll ret = a * b - M * ull(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {
    ull ans = 1;
    for (; e; b = modmul(b, b, mod), e /= 2)
        if (e & 1) ans = modmul(ans, b, mod);
    return ans;
}
```

ModSqrt.h
Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).
Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

```
"ModPow.h"
ll sqrt(ll a, ll p) {
    a %= p; if (a < 0) a += p;
    if (a == 0) return 0;
    assert(modpow(a, (p-1)/2, p) == 1); // else no solution
    if (p % 4 == 3) return modpow(a, (p+1)/4, p);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
    ll s = p - 1, n = 2;
    int r = 0, m;
    while (s % 2 == 0)
        ++r, s /= 2;
    while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
    ll x = modpow(a, (s + 1) / 2, p);
    ll b = modpow(a, s, p), g = modpow(n, s, p);
    for (; r = m) {
        ll t = b;
        for (m = 0; m < r && t != 1; ++m)
            t = t * t % p;
        if (m == 0) return x;
    }
```

```
ll gs = modpow(g, 1LL << (r - m - 1), p);
g = gs * gs % p;
x = x * gs % p;
b = b * g % p;
}
}
```

2.2 Primality
FastEratosthenes.h
Description: Prime sieve for generating all primes smaller than LIM.
Time: LIM=1e9 $\approx$ 1.5s

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
    const int S = (int)round(sqrt(LIM)), R = LIM / 2;
    vi pr = {2}, sieve(S+1); pr.reserve(int(LIM/log(LIM)*1.1));
    vector<pii> cp;
    for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
        cp.push_back({i, i * i / 2});
        for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
    }
    for (int L = 1; L <= R; L += S) {
        array<bool, S> block{};
        for (auto &[p, idx] : cp)
            for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
        rep(i,0,min(S, R - L))
            if (!block[i]) pr.push_back((L + i) * 2 + 1);
    }
    for (int i : pr) isPrime[i] = 1;
    return pr;
}
```

MillerRabin.h
Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \bmod c$.

```
"ModMulLL.h"
bool isPrime(ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
        s = __builtin_ctzll(n-1), d = n >> s;
    for (ull a : A) {
        ull p = modpow(a%n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
            p = modmul(p, p, n);
        if (p != n-1 && i != s) return 0;
    }
    return 1;
}
```

Factor.h
Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}(n^{1/4})$, less for numbers with small factors.

```
"ModMulLL.h", "MillerRabin.h"
ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return modmul(x, x, n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = modmul(prd, max(x,y) - min(x,y), n)) prd = q;
            x = f(x), y = f(y));
    }
    return __gcd(prd, n);
}
```

```
vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}
```

2.3 Divisibility
euclid.h
Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in **__gcd** instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

```
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a/b * x, d;
}
```

CRT.h
Description: Chinese Remainder Theorem.
crt(a, m, b, n) computes x such that $x \equiv a \pmod m$, $x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

```
"euclid.h"
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m*n/g : x;
}
```

2.3.1 Bézout's identity
For $a \neq, b \neq 0$, then $d = \gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h
Description: Euler's ϕ function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1$, p prime $\Rightarrow \phi(p^k) = (p - 1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} \dots p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1} \dots (p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n} (1 - 1/p)$. $\sum_{d|n} \phi(d) = n$, $\sum_{1 \leq k \leq n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$.
Euler's thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Fermat's little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p \forall a$.

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
    rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
    for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
        for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

2.4 Fractions

ContinuedFractions.h

Description: Given N and a real number $x \geq 0$, finds the closest rational approximation p/q with $p, q \leq N$. It will obey $|p/q - x| \leq 1/qN$.

For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. (p_k/q_k alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞ ; if x is the root of a degree 2 polynomial the a 's eventually become cyclic.

Time: $\mathcal{O}(\log N)$

dd6c5e, 21 lines

```
typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximate(d x, ll N) {
    ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
    for (;;) {
        ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf),
            a = (ll)floor(y), b = min(a, lim),
            NP = b*P + LP, NQ = b*Q + LQ;
        if (a > b) {
            // If b > a/2, we have a semi-convergent that gives us a
            // better approximation; if b = a/2, we *may* have one.
            // Return {P, Q} here for a more canonical approximation.
            return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
                make_pair(NP, NQ) : make_pair(P, Q);
        }
        if (abs(y = 1/(y - (d)a)) > 3*N) {
            return {NP, NQ};
        }
        LP = P; P = NP;
        LQ = Q; Q = NQ;
    }
}
```

FracBinarySearch.h

Description: Given f and N , finds the smallest fraction $p/q \in [0, 1]$ such that $f(p/q)$ is true, and $p, q \leq N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.

Usage: `fracBS([](Frac f) { return f.p>=3*f.q; }, 10);` // {1,3}

Time: $\mathcal{O}(\log(N))$

27ab3e, 25 lines

```
struct Frac { ll p, q; };
```

```
template<class F>
Frac fracBS(F f, ll N) {
    bool dir = 1, A = 1, B = 1;
    Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
    if (f(lo)) return lo;
    assert(f(hi));
    while (A || B) {
        ll adv = 0, step = 1; // move hi if dir, else lo
        for (int si = 0; step; (step *= 2) >= si) {
            adv += step;
            Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
            if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
                adv -= step; si = 2;
            }
        }
        hi.p += lo.p * adv;
        hi.q += lo.q * adv;
        dir = !dir;
        swap(lo, hi);
        A = B; B = !!adv;
    }
    return dir ? hi : lo;
}
```

2.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

ContinuedFractions FracBinarySearch

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

2.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a , except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

2.7 Estimates

$$\sum_{d|n} d = O(n \log \log n).$$

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

2.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g(n/d)$$

Other useful formulas/forms:

$$\sum_{d|n} \mu(d) = [n = 1] \text{ (very useful)}$$

$$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n)g(d)$$

$$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m)g(\lfloor \frac{n}{m} \rfloor)$$